

Link del repositorio:

https://github.com/Eric0298/practica_jenkins.git

Pràctica Jenkins

Utilitzant un projecte React qualsevol, haureu de crear una nova branca (ci_jenkins) en la qual existirà un jenkinsfile per a poder executar una pipeline amb les següents característiques:

(0,3 punts) Instal·lar el plugin Build Monitor View (link) i configurar una vista en la qual es mostren totes les tasques executades en Jenkins

(0,5 punts). En executar-se, en una primera Stage ('Petició de dades') demanarà tres valors per pantalla:

Executor: en el qual s'especificarà el nom de la persona que executa la pipeline

Motiu: en què podem especificar el motiu pel qual estem executant la pipeline.

Chat ID: Emmagatzemarà el ChatID de telegram al qual notificarem el resultat de cada stage executat

(1 punt) Stage "Linter". S'encarregarà d'executar un linter sobre el projecte react utilitzant la llibreria del següent link (de no disposar d'ell, haureu d'instal·lar-lo i configurar-lo). En cas que existisquen errors haureu de corregir-los fins que el job s'execute sense problemes.

(1 punt) Stage "Test". El vostre projecte haurà d'incloure almenys 3 tests implementats amb jest en els quals verificàreu la funcionalitat de tres mètodes del vostre codi (mètodes que s'utilitzen en la lògica de l'aplicació)

(0,3 punts) Stage "Build" realitzarà el build del projecte per a tindre una versió empaquetada del mateix que serà la que acabarà publicant-se en Vercel.

(1 punto) Stage "Update_Readme". Ejecutará un script (que deberéis crear dentro de una carpeta jenkinsScripts en vuestro proyecto) que se encargará de modificar el README.md del proyecto añadiendo uno de los siguientes badges al final del mismo y tras el texto "RESULTADO DE LOS ÚLTIMOS TESTS":

(Failure) <https://img.shields.io/badge/test-failure-red>

(Success) <https://img.shields.io/badge/tested%20with-Cypress-04C38E.svg>

(1 punt) Stage “Push_Changes”. Executarà un script (que també estarà dins de la carpeta jenkinsScripts) encarregat d'executar el add, commit i push dels canvis del readme al nostre repositori de codi. El comentari del commit haurà de seguir el següent format:

Pipeline executada per PARAM_EXECUTOR. Motiu: PARAM_MOTIU

(1,5 punts) Stage “Deploy to Vercel”. Executarà un script (que també estarà dins de la carpeta jenkinsScripts) encarregat de publicar el projecte en la plataforma vercel (link). S'executarà només si la resta de Stages han sigut satisfactòries.

PISTA: podeu utilitzar el CLI de vercel (link) per a executar el desplegament de manera fàcil

(1 punts) Stage “Notificació”. S'encarregarà d'enviar un missatge al bot de telegram amb:

- Destinatar: chatId establida com a paràmetre
- missatge:

S'ha executat la pipeline de jenkins amb els següents resultats:

- Linter_stage: resultat associat
- Test_stage: resultat associat
- Update_readme_stage: resultat associat
- Deploy_to_Vercel_stage: resultat associat

El resultat associat de cada stage s'emmagatzemarà utilitzant variables d'entorn.

Qualsevol variable el valor de la qual puga ser comprometedor o secret, haurà de configurar-se com una credencial de Jenkins.

Una vegada realitzades totes les accions anteriors, documenteu correctament tots els passos realitzats en el Readme.md del projecte. Per descomptat, haurà d'incloure una introducció teòrica dels conceptes tractats (jenkins principalment) així com el procés de configuració de la pipeline a través de la UI de jenkins (1,5 punts)

Com a resposta a l'activitat plantejada, entregareu el link al vostre repositori github així com el link de vercel on estarà desplegat el projecte.

1.Empezamos ejecutando el Docker-compose.yml que nos han adjuntado en aules para trabajar con la imagen de Jenkins.

```
C:\Users\Usuario\jenkins>docker-compose up -d
time="2025-01-10T18:57:46+01:00" level=warning msg="C:\\Users\\Usuario\\jenkins\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 1/1
✓ Container jenkins-server Started 0.3s
```

2.Entramos en el localhost:8080 donde tenemos ubicado el Jenkins

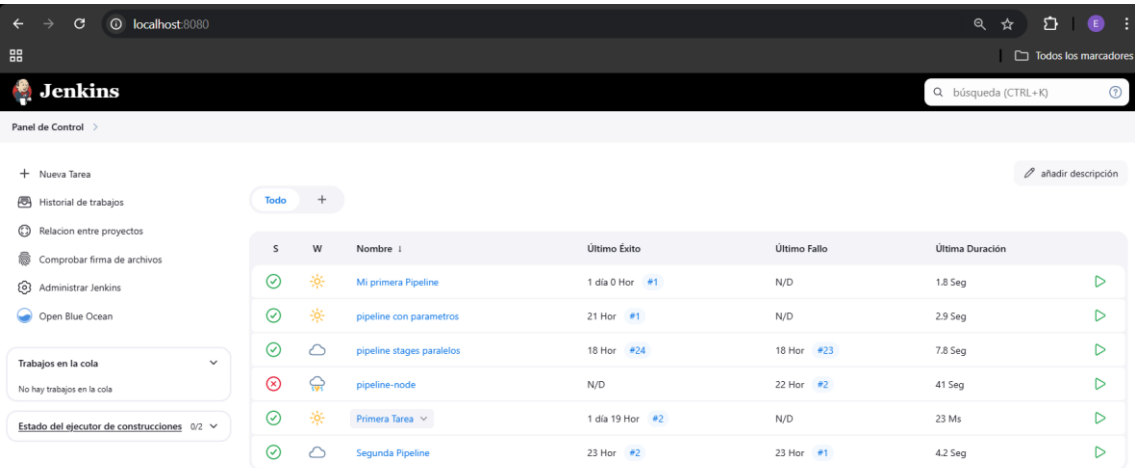
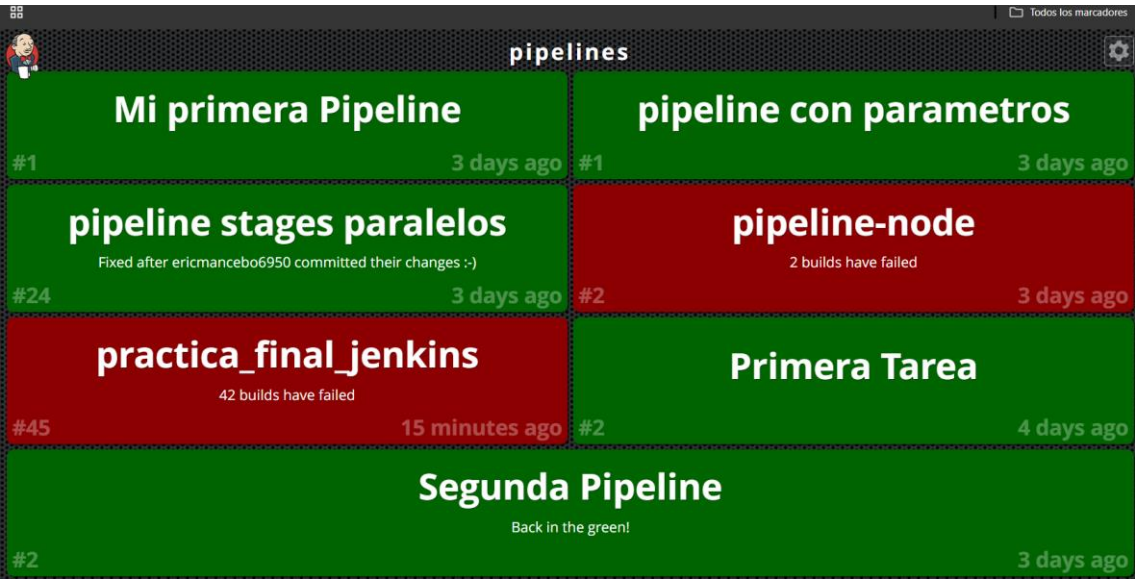


IMAGEN DEL LISTADO DE TAREAS CON EL PLUGGIN



3. Una vez corriendo el servidor Jenkins iniciaremos un nuevo repositorio de GitHub.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Eric0298 / Repository name * practica_jenkins
 ✔ practica_jenkins is available.

Great repository names are short and memorable. Need inspiration? How about [turbo-adventure](#) ?

Description (optional)

☒ **Public**
 Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
 You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file
 This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore
 .gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Una vez creado el repositorio, lo clonaremos para trabajar en local.

```
Usuario@DESKTOP-560EG10 MINGW64 ~ (main)
$ cd Documents/'DAW SEMI'/2DAW/Despliegue

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue (main)
$ cd UD4_Jenkins/PracticaFinal

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (main)
$ ls
'Practica Jenkins.docx'  '~$actica Jenkins.docx'

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (main)
$ git clone https://github.com/Eric0298/practica_jenkins.git
Cloning into 'practica_jenkins'...
warning: You appear to have cloned an empty repository.
```

Trabajaremos sobre la rama especificada en la practica (ci_jenkins) , asi que la crearemos y estableceremos el remoto en el nuevo repositorio.

```
Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (main)
$ git checkout -b ci_jenkins
Switched to a new branch 'ci_jenkins'

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (ci_jenkins)
$ git remote -v
origin C:/Users/Usuario/repoControlsAvançats (fetch)
origin C:/Users/Usuario/repoControlsAvançats (push)

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (ci_jenkins)
$ git remote set-url origin https://github.com/Eric0298/practica_jenkins.git

Usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/PracticaFinal (ci_jenkins)
$ git remote -v
origin https://github.com/Eric0298/practica_jenkins.git (fetch)
origin https://github.com/Eric0298/practica_jenkins.git (push)
```

una vez establecido el entorno de trabajo empezamos con la practica

4. Creamos un proyecto node yreact en la raíz del proyecto.

```

$ npm install
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you
more comprehensive and powerful.
npm warn deprecated @babel/plugin-proposal-private-methods@7.18.6: This proposal has been merged to the ECMAScript standard a
ite-methods instead.
npm warn deprecated @babel/plugin-proposal-nullish-coalescing-operator@7.18.6: This proposal has been merged to the ECMAScrip
nsform-nullish-coalescing-operator instead.
npm warn deprecated @babel/plugin-proposal-numeric-separator@7.18.6: This proposal has been merged to the ECMAScript standard
eric-separator instead.
npm warn deprecated @babel/plugin-proposal-class-properties@7.18.6: This proposal has been merged to the ECMAScript standard
s-properties instead.
npm warn deprecated @humanwhocodes/config-array@0.13.0: Use @eslint/config-array instead
npm warn deprecated stable@0.1.8: Modern JS already guarantees Array#sort() is a stable sort, so this library is deprecated.
Script/Reference/Global_Objects/Array/sort#browser_compatibility
npm warn deprecated @babel/plugin-proposal-optional-chaining@7.21.0: This proposal has been merged to the ECMAScript standard
ional-chaining instead.
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
npm warn deprecated rollup-plugin-terser@7.0.2: This package has been deprecated and is no longer maintained. Please use @ro
npm warn deprecated abab@2.0.6: Use your platform's native atob() and btoa() methods instead
npm warn deprecated q@1.5.1: You or someone you depend on is using Q, the JavaScript Promise library that gave JavaScript de
ve JavaScript promise now. Thank you literally everyone for joining me in this bet against the odds. Be excellent to each oth
npm warn deprecated
npm warn deprecated (For a CapTP with native promises, see @endo/eventual-send and @endo/captp)
npm warn deprecated @humanwhocodes/object-schema@2.0.3: Use @eslint/object-schema instead
npm warn deprecated domexception@2.0.1: Use your platform's native DOMException instead
npm warn deprecated sourcemap-codec@1.4.8: Please use @jridgewell/sourceemap-codec instead
npm warn deprecated w3c-hr-time@1.0.2: Use your platform's native performance.now() and performance.timeOrigin.
npm warn deprecated workbox-cacheable-response@6.6.0: workbox-background-sync@6.6.0
npm warn deprecated workbox-google-analytics@6.6.0: It is not compatible with newer versions of GA starting with v4, as long
npm warn deprecated svgo@1.3.2: This SVGO version is no longer supported. Upgrade to v2.x.x.
npm warn deprecated eslint@8.57.1: This version is no longer supported. Please see https://eslint.org/version-support for otl

added 1323 packages, and audited 1324 packages in 1m

267 packages are looking for funding
  run `npm fund` for details

```

5. Ahora estableceremos el entorno de desarrollo, instalaremos ESLint y lo inicializaremos.

2.2 Instalamos jest y lo configuramos.

```

usuario@DESKTOP-56OEG10 MINGW64 ~/Documents/DAW_SEMI/2DAW/Despliegue/UD4_Jenkins/practica_final/practica_jenkins (ci_jenkins)
$ npm install jest --save-dev
up to date, audited 1339 packages in 13s
278 packages are looking for funding
  run 'npm fund' for details
8 vulnerabilities (2 moderate, 6 high)
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.

usuario@DESKTOP-56OEG10 MINGW64 ~/Documents/DAW_SEMI/2DAW/Despliegue/UD4_Jenkins/practica_final/practica_jenkins (ci_jenkins)
$ npm install @testing-library/react @testing-library/jest-dom --save-dev
added 16 packages, and audited 1355 packages in 4s
278 packages are looking for funding
  run 'npm fund' for details
8 vulnerabilities (2 moderate, 6 high)
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.

```

- Modificamos el package.json y añadimos en los scripts el test por jest.
Para evitar conflictos con react crearemos el script test:jest

```

"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject",
  "test:jest": "jest"
},

```

- Este script lo deberemos tener cuenta a la hora de configurar la pipeline de Jenkins file

2.2.1 Vamos a crear los tests para ejecutar el jest en nuestro proyecto :

creamos una carpeta tests en el src del proyecto y creamos trest test sencillos para que los pase juntos con sus respectivos scripts:

test 1

```

src > _tests_ > RenderText.test.js > test('RenderText muestra el texto correcto') callback
1  import React from 'react';
2  import { render, screen } from '@testing-library/react';
3  import RenderText from '../components/RenderText';
4
5  test('RenderText muestra el texto correcto', () => {
6    render(<RenderText />);
7    // Verifica si el texto esperado aparece en el DOM
8    expect(screen.getByText('Has pasado el test')).toBeInTheDocument();
9  });
10

```

Test 2.

```
RenderText.test.js U package.json M RenderLogo.test.js U X RenderLink.test.js U
src > __tests__ > RenderLogo.test.js > ...
1 import React from 'react';
2 import { render, screen } from '@testing-library/react';
3 import RenderLogo from '../components/RenderLogo';
4
5 test('RenderLogo muestra la imagen correctamente', () => {
6   render(<RenderLogo />);
7
8   // Verifica que la imagen esté presente en el DOM
9   const logoElement = screen.getByAltText('Logo de ejemplo');
10  expect(logoElement).toBeInTheDocument();
11
12  // Verifica que la imagen tenga la URL correcta
13  expect(logoElement).toHaveAttribute('src', 'logo.png');
14 });
15
```

Test 3.

```
1 import React from 'react';
2 import { render, screen } from '@testing-library/react';
3 import RenderLink from '../components/RenderLink';
4
5 test('RenderLink muestra el enlace correcto', () => {
6   render(<RenderLink />);
7
8   // Verifica si el enlace tiene el texto correcto
9   const linkElement = screen.getByText('Haz clic aquí');
10  expect(linkElement).toBeInTheDocument();
11
12  // Verifica si el enlace tiene el href correcto
13  expect(linkElement).toHaveAttribute('href', 'https://www.ejemplo.com');
14 });
15
```

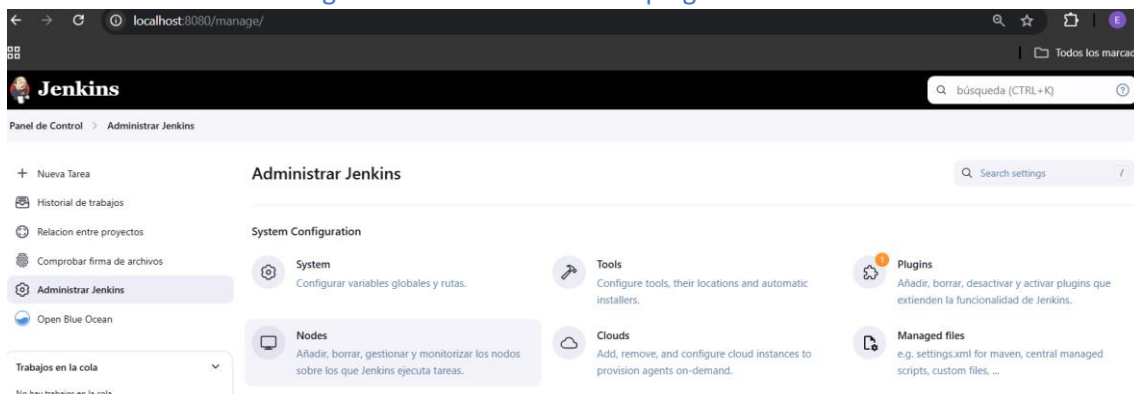
2.2.2 Ejecutamos los tests para ver si hay fallos

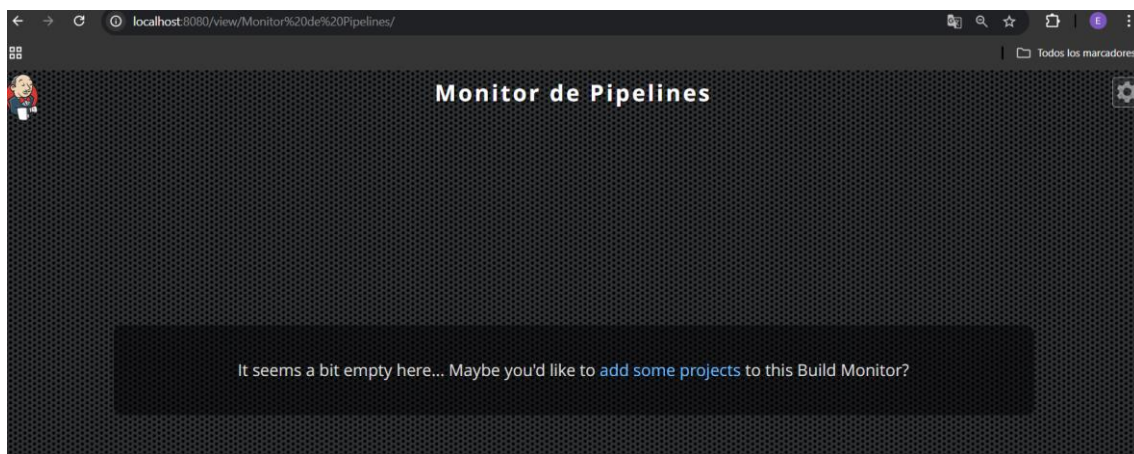
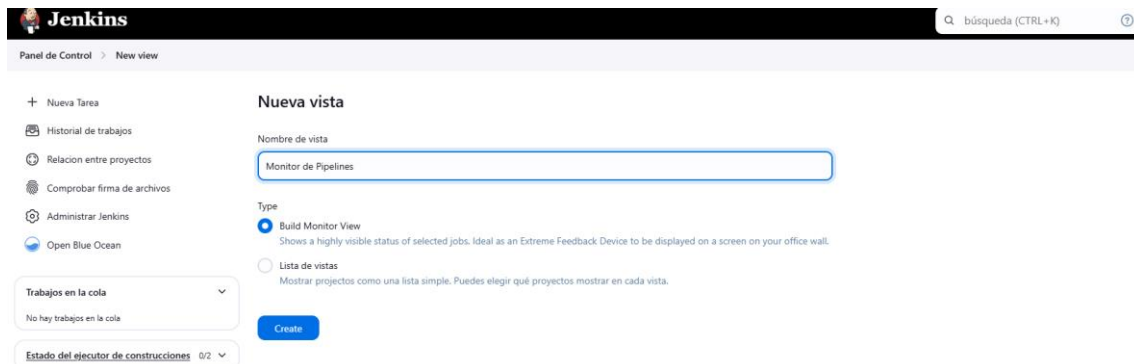
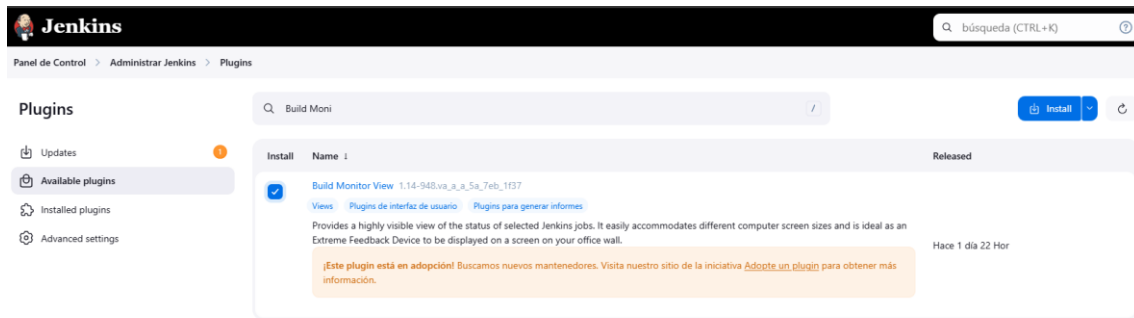
```
usuario@DESKTOP-560EG10 MINGW64 ~/Documents/DAW SEMI/2DAW/Despliegue/UD4_Jenkins/practica_final/practica_jenkins (ci_jenkins)
$ npm run test:jest
> practica_jenkins@0.1.0 test:jest
> jest

PASS src/__tests__/RenderLink.test.js
PASS src/__tests__/RenderLogo.test.js
PASS src/__tests__/RenderText.test.js

Test Suites: 3 passed, 3 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 1.845 s, estimated 2 s
Ran all test suites.
```

3. Ahora deberíamos configurar Jenkins instalando el plugin de Build Monitor Views





- Ahora se muestra así porque no hemos creado el proyecto en Jenkins

4. Ahora creamos el jenkinsfile:

```

Usuario@DESKTOP-560EGI0 MINGW64 ~/Documents/DAW
SEMI/2DAW/Despliegue/UD4_Jenkins/practica_final/practica_jenkins (ci_jenkins)
$ cat Jenkinsfile
pipeline {
  agent any
  tools { nodejs "Node" }
  environment {
    TELEGRAM_BOT_TOKEN = credentials('telegram_bot_token')
    TELEGRAM_CHAT_ID = credentials('telegram_chat_id')
    EXECUTOR = ''
    MOTIVO = ''
    CHAT_ID = ''
    LINTER_RESULT = ''
    TEST_RESULT = ''
    BUILD_RESULT = ''
  }
}

```



```

    UPDATE_README_RESULT = ''
    DEPLOY_RESULT = 'NOT_EXECUTED' // valor predeterminado
  }
  parameters {
    string(name: 'EXECUTOR', defaultValue: '', description: 'Nombre de la
persona ejecutando la pipeline')
    string(name: 'MOTIVO', defaultValue: '', description: 'Motivo para
ejecutar la pipeline')
    string(name: 'CHAT_ID', defaultValue: '', description: 'Chat ID de
Telegram para las notificaciones')
  }
  stages {
    stage('Clean Workspace') {
      steps {
        cleanWs()
      }
    }

    stage('Checkout') {
      steps {
        script {
          checkout([$class: 'GitSCM',
                    branches: [[name: '*/ci_jenkins']],
                    userRemoteConfigs: [[url:
'https://github.com/Eric0298/practica_jenkins.git']]
          ])
        }
      }
    }

    stage('Petició de dades') {
      steps {
        script {
          EXECUTOR = params.EXECUTOR
          MOTIVO = params.MOTIVO
          CHAT_ID = params.CHAT_ID
          echo "Datos recibidos: Ejecutando por ${EXECUTOR}, motivo:
${MOTIVO}, chat ID: ${CHAT_ID}"
        }
      }
    }

    stage('Install Dependencies') {
      steps {
        script {
          echo "Instalando dependencias..."
          sh 'npm install'
        }
      }
    }

    stage('Linter') {
      steps {
        script {
          echo "Ejecutando linter..."
          sh 'npm run lint'
          LINTER_RESULT = currentBuild.result
        }
      }
    }

    stage('Test') {
      steps {
        script {
          echo "Ejecutando tests..."
          sh 'npm run test:jest'
          TEST_RESULT = currentBuild.result
        }
      }
    }

    stage('Build') {
      steps {
        script {
          echo "Construyendo proyecto..."
          sh 'npm run build'
          BUILD_RESULT = currentBuild.result
        }
      }
    }
  }
}

```

```

    }
  }
}

stage('Check Permissions') {
  steps {
    script {
      echo "Verificando permisos de los scripts..."
      sh 'ls -l ./jenkinsScripts/'
    }
  }
}

stage('Update_Readme') {
  steps {
    script {
      echo "Asignando permisos de ejecución al script..."
      sh 'chmod +x ./jenkinsScripts/updateReadme.sh'
      echo "Actualizando README..."
      sh './jenkinsScripts/updateReadme.sh'
      UPDATE_README_RESULT = currentBuild.result
    }
  }
}

stage('Push_Changes') {
  steps {
    script {
      echo "Configurando identidad de Git y enviando cambios..."
      sh 'chmod +x ./jenkinsScripts/pushChanges.sh'
      withCredentials([usernamePassword(credentialsId:
'680e2c18-0bce-4ff0-b6f0-7e4cd45bf25d', usernameVariable: 'GIT_USERNAME',
passwordvariable: 'GIT_PASSWORD')]) {
        sh """
            git config credential.helper 'store'
            echo
'https://${GIT_USERNAME}:${GIT_PASSWORD}@github.com' > ~/.git-credentials
            sh './jenkinsScripts/pushChanges.sh "${EXECUTOR}"
"${MOTIVO}"
        """
      }
    }
  }
}

stage('Deploy to Vercel') {
  when {
    expression { return BUILD_RESULT == 'SUCCESS' }
  }
  steps {
    script {
      echo "Desplegando a Vercel..."
      withCredentials([string(credentialsId: 'vercel_token',
variable: 'VERCEL_TOKEN')]) {
        sh "vercel --token $VERCEL_TOKEN --prod"
      }
      DEPLOY_RESULT = currentBuild.result
    }
  }
}

stage('Notificación') {
  steps {
    script {
      echo "Enviando notificación a Telegram..."
      def deployStatus = DEPLOY_RESULT == 'SUCCESS' ? 'Éxito' :
(DEPLOY_RESULT == 'NOT_EXECUTED' ? 'No ejecutado' : 'Fallo')
      sh """
        ./jenkinsScripts/sendNotification.sh \
        ${TELEGRAM_CHAT_ID} \
        ${LINTER_RESULT} \
        ${TEST_RESULT} \
        ${UPDATE_README_RESULT} \
        ${deployStatus}
      """
    }
  }
}

```

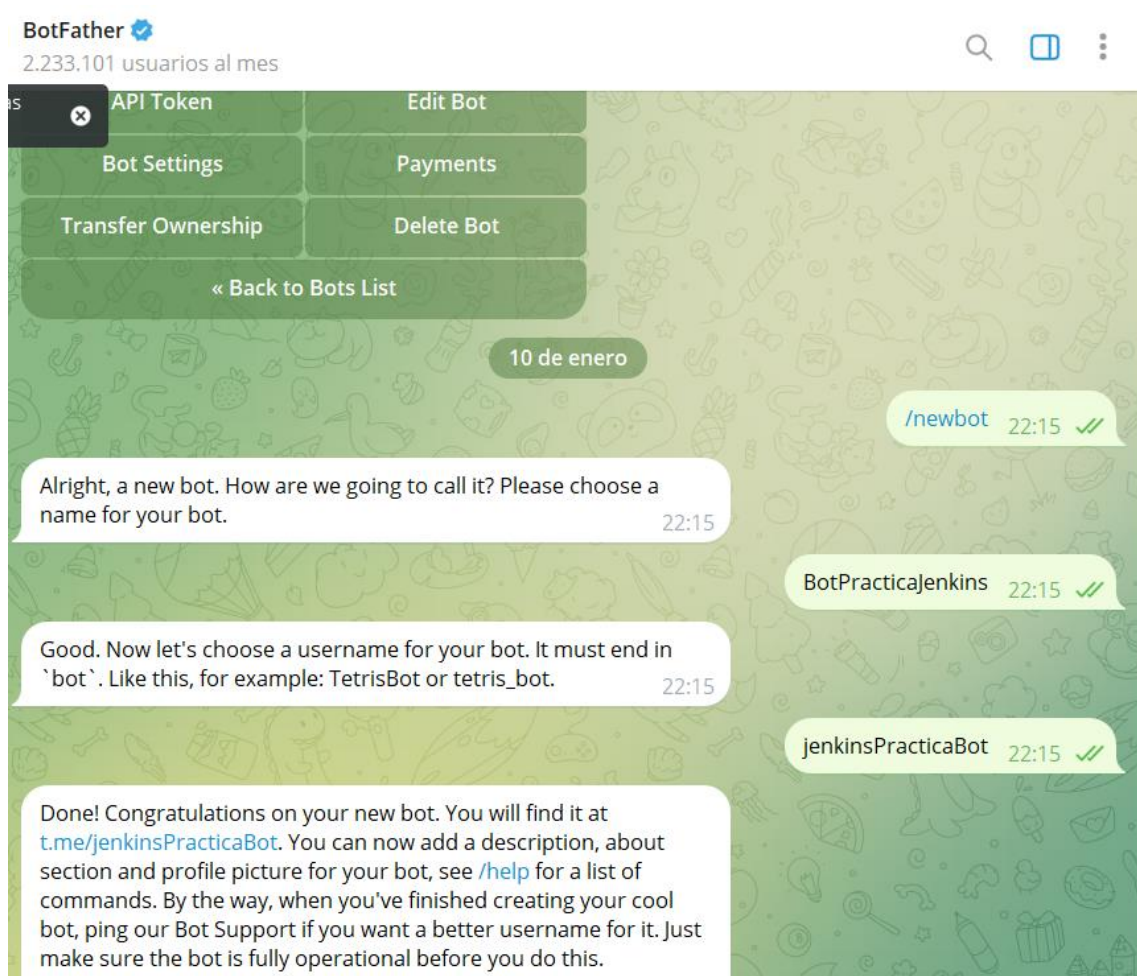
```

    }
  }
  post {
    always {
      echo "Pipeline ejecutada por ${EXECUTOR} con motivo: ${MOTIVO}"
    }
  }
}

```

Pasos previos importantes para la configuración de los scripts de Jenkins

- Crear el bot de telegram y gestionar los tokens.



-Obtenemos el id del bot:

* Iniciamos conversación con el bot creado

*Abrimos el navegador y buscamos : <https://api.telegram.org/bot> + el token que nos ha dado anteriormente+/getUpdates y te respondera con la info en formato json

```

1 {
2   "ok": true,
3   "result": [
4     {
5       "update_id": 507877100,
6       "message": {
7         "message_id": 3,
8         "from": {
9           "id": 5507344445,
10          "is_bot": false,
11          "first_name": "Eric",
12          "last_name": "Mancebo",
13          "username": "EricMancebo",
14          "language_code": "es"
15        },
16        "chat": {
17          "id": 5507344445,
18          "first_name": "Eric",
19          "last_name": "Mancebo",
20          "username": "EricMancebo",
21          "type": "private"
22        },
23        "date": 1736544555,
24        "text": "hola bot"
25      }
26    ]
27  }
28 }

```

*En Jenkins creamos las credenciales como secrets para guardar la infor del chatID y el token que nos ha dado bothfather.

Administrar Jenkins

System Configuration

- System**: Configurar variables globales y rutas.
- Tools**: Configure tools, their locations and automatic installers.
- Plugins**: Añadir, borrar, desactivar y activar plugins que extienden la funcionalidad de Jenkins.
- Nodes**: Añadir, borrar, gestionar y monitorizar los nodos sobre los que Jenkins ejecuta tareas.
- Clouds**: Add, remove, and configure cloud instances to provision agents on-demand.
- Managed files**: e.g. settings.xml for maven, central managed scripts, custom files, ...

Security

- Security**: Seguridad en Jenkins. Define quién tiene acceso al sistema (autenticación) y qué puede hacer (autorización).
- Credentials**: Configure credentials
- Credential Providers**: Configure the credential providers and types

Panel de Control > Administrar Jenkins > Credentials > System > Global credentials (unrestricted)

Global credentials (unrestricted) [+ Add Credentials](#)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
680e2c18-0bce-4ff0-b6d0-7e4cd45bf25d	Eric0298/*****	Username with password	

Icono: S M L

Panel de Control > Administrar Jenkins > Credenciales > System > Global credentials (unrestricted) >

New credentials

Kind
Secret text

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

Secret
.....

ID ?
telegram_bot_token

Description ?

Create

- Una vez creado el Jenkinsfile creamos la carpeta de scriptsJenkins donde ubicaremos los scripts.

```

1  #!/bin/bash
2  if [ "$1" == "success" ]; then
3      echo "RESULTADO DE LOS ÚLTIMOS TESTS: ![Success](https://img.shields.io/badge/tested%20with-cypress-04C38E.svg)" >> README.md
4  else
5      echo "RESULTADO DE LOS ÚLTIMOS TESTS: ![Failure](https://img.shields.io/badge/test-failure-red)" >> README.md
6  fi
7

```

```

1  #!/bin/bash
2  git config --global user.email "ericmancebo6950@gmail.com"
3  git config --global user.name "$1"
4  git add README.md
5  git commit -m "Pipeline ejecutada por $1. Motiu: $2"
6  git push origin ci_jenkins
7

```

```

1  #!/bin/bash
2  vercel --prod
3

```

```

1  #!/bin/bash
2
3  # Recuperar el token y chat_id desde las credenciales de Jenkins
4  BOT_TOKEN=$(cat /var/jenkins_home/creds/telegram_bot_token)
5  CHAT_ID=$(cat /var/jenkins_home/creds/telegram_chat_id)
6
7  # Enviar el mensaje a Telegram
8  curl -X POST https://api.telegram.org/bot${BOT_TOKEN}/sendMessage \
9  -d chat_id=${CHAT_ID} \
10 -d text="Pipeline ejecutada con los siguientes resultados:
11 - Linter: $2
12 - Test: $3
13 - Update Readme: $4
14 - Deploy to Vercel: $5"
15

```

- Hacemos push con los cambios

- creamos la tarea en Jenkins

Nuevo Tarea

Enter an item name

practica_final_jenkins

Select an item type



Crear un proyecto de estilo libre

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Gestiona actividades de larga duración que pueden abarcar varios agentes de construcción. Apropiado para construir pipelines (conocidas anteriormente como workflows) y/o para la organización de actividades complejas que no se pueden articular fácilmente con tareas de tipo freestyle.



Crear un proyecto multi-configuración

Adecuado para proyectos que requieran un gran número de configuraciones diferentes, como testear en multiples entornos, ejecutar sobre plataformas concretas, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline objects according to detected branches in one SCM repository.

OK

Panel de Control > practica_final_jenkins > Configuration

Configure

Plain text [Visualizar](#)

- ☐ Desechar ejecuciones antiguas ?
- ☐ Do not allow concurrent builds
- ☐ Do not allow the pipeline to resume if the controller restarts
- ☒ Esta ejecución debe parametrizarse ?

Añadir un parámetro ^

- ▼ Filter
- Credentials Parameter
- Elección
- Parámetro de cadena
- Parámetro de contraseña
- Parámetro de ejecución
- Parámetro de fichero
- Parámetro de texto
- Valor booleano

- ☐ Ejecutar periódicamente ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Consultar repositorio (SCM) ?

añadimos todos los parámetros creados en el jenkinsfile

☒ Esta ejecución debe parametrizarse ?

Parámetro de texto

Name ?

EXECUTOR

Default Value ?

Description ?

Nombre de la persona ejecutando la pipeline

Plain text [Visualizar](#)

Pipeline

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Eric0298/practica_jenkins.git

Credentials ?

Eric0298/*****

+ Add

Avanzado ▼

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/ci_jenkins

8. Construimos el proyecto:

Status

Changes

Build with Parameters

Configurar

Borrar Pipeline

Open Blue Ocean

Rename

Pipeline Syntax

Pipeline practica_final_jenkins

Esta ejecución requiere parámetros adicionales:

EXECUTOR
Nombre de la persona ejecutando la pipeline

ERIC

MOTIVO
Motivo para ejecutar la pipeline

PRUEBA

CHAT_ID
Chat ID de Telegram para las notificaciones

Ejecución Cancel

practica_final_jenkins < 45

Pipeline Modificación Pruebas Artefacto

Rama: 28s Modificado por ericmancebo6950

Commit: 19 minutos ago Lanzada por el usuario anonymous

Start Clean Workspace Checkout Petición de datos Install Dependencies Linter Test Build Check Permissions Update_Readme Push_Changes Deploy to Vercel Notificación End

Notificación - <1s

Restart Notificación

- Node - Use a tool from a predefined Tool Installation <1s
- Fetches the environment variables for a given tool in a list of 'FOO-bar' strings suitable for the withEnv step. <1s
- Enviando notificación a Telegram... - Print Message <1s
- ./jenkinsScripts/sendNotification.sh \${TELEGRAM_CHAT_ID} null null null No ejecutado - Shell Script <1s
- Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure. Affected argument(s) used the following variable(s): [TELEGRAM_CHAT_ID]. See https://jenkins.io/redirect/groovy-string-interpolation for details. <1s
- ./jenkinsScripts/sendNotification.sh ***** null null null No ejecutado <1s
- script returned exit code 126 <1s
- Pipeline ejecutada por ERIC con motivo: PRUEBA - Print Message <1s